

ATOMiK — Due Diligence Follow-Up

To: Chris Bolt From: Matt Rockwell, Founder Date: March 11, 2026

Chris,

Thank you for the thorough feedback. This document restructures around the questions you raised, leading with the problem and commercial case rather than the architecture.

The Problem

Real-time embedded systems — sensor fusion pipelines, industrial controllers, autonomous navigation stacks — repeatedly copy and reconstruct large state structures even when only small portions change. A sensor fusion node tracking 50 inputs doesn't need to move the entire state buffer every cycle. It needs to track what changed and reconstruct on demand.

This is not a niche inefficiency. It is the default behavior of every embedded system built on conventional memory architectures. The result: wasted memory bandwidth, wasted power, and unpredictable latency from cache effects and speculative execution. In battery-powered edge devices, bandwidth and power are the binding constraints. In safety-critical systems, non-deterministic latency is a disqualifier.

The market is large and growing. Edge AI and IoT alone represent a \$50B+ semiconductor TAM, with memory subsystem IP as a meaningful slice. But the relevant observation is simpler: every real-time system that tracks changing state has this problem, and while caching, compression, and log-structured storage reduce some overhead, no existing architecture addresses the problem by accumulating state deltas directly in hardware.

The Solution

ATOMiK is a hardware IP core that replaces full-state memory operations with delta-state accumulation. Instead of reading and writing entire state buffers, it tracks changes as XOR deltas and reconstructs current state on demand: $\text{current_state} = \text{initial_state} \oplus \text{accumulated_deltas}$.

This is not a cache optimization or a compression scheme. It is a different execution model — one where the cost of a state update is proportional to what changed, not to the size of the state.

What it delivers: - **120x-30,720x memory traffic reduction** across 9 validated workloads (360 measurements, statistical significance confirmed) - **Deterministic latency** — no caches, no speculation, no data-dependent timing variation. 2-cycle jitter or less on hardware. - **Tiny footprint** — 287 LUT on Xilinx Zynq (0.54% of device), 1.8 mW. Embeds alongside existing customer logic with negligible resource cost. - **Vendor-portable** — same RTL synthesizes on Xilinx and Gowin with zero changes. Lattice and Intel are straightforward targets.

The mathematical foundation is formally proven: 92 theorems in Lean4 establish that XOR accumulation over arbitrary bit vectors forms an Abelian group. The proofs are machine-checked with zero unproven axioms. This is not “we tested a lot of cases” — the properties hold for all possible inputs by construction.

The convergence of inexpensive FPGA platforms, formal verification tooling, and schema-driven code generation makes this architecture commercially viable today in a way it was not even five years ago.

Where It Doesn't Work

ATOMiK's advantage depends on the read/write ratio. It wins on write-heavy workloads (55% faster at 90% writes) and loses on read-heavy ones (32% slower at 90% reads). The crossover is around 50%. It is a purpose-built accelerator for state-tracking workloads, not a general-purpose memory replacement.

Why This Hasn't Been Done Before

Three barriers kept this design space unexplored:

- 1. XOR accumulation looks trivial until you formalize it.** The individual operation is simple. The insight that XOR over fixed-width vectors forms a complete algebraic group — with commutativity enabling lock-free parallel accumulation, self-inverse enabling instant rollback, and identity enabling zero-cost initialization — requires mathematical formalization that the embedded systems community has not pursued. The 92 Lean4 proofs are not an academic exercise; they establish properties that make the hardware architecture correct by construction.
- 2. FPGA accessibility was too limited.** Until recently, deploying custom IP cores required expensive dev boards, proprietary toolchains, and months of integration work. The maturation of low-cost SoC platforms (Zynq, Lattice Nexus) and open-source FPGA workflows means a novel IP core can now be evaluated in a customer's real design environment for <\$200 in hardware. This was not practical a decade ago.
- 3. No schema-driven SDK existed to bridge the gap.** A hardware delta-state core is useful only if software developers can integrate it without understanding XOR algebra. ATOMiK's SDK takes a JSON schema describing the developer's state structure and generates typed libraries in 5 languages (Python, C, Rust, Go, TypeScript) with automatic delta computation, change detection, and type-safe field access.

353 tests pass across all targets. This SDK layer is what makes the hardware commercially viable — without it, each customer integration would require custom engineering.

The combination of formal verification, low-cost FPGA platforms, and schema-driven code generation is what makes ATOMiK buildable and deployable now, by a small team, at seed-stage capital levels.

The Developer Experience

Developers define their state structure as a JSON schema:

```
{
  "name": "SensorFusionState",
  "fields": [
    {"name": "accel_x", "type": "float64"},
    {"name": "accel_y", "type": "float64"},
    {"name": "gyro_z", "type": "float64"},
    {"name": "timestamp", "type": "uint64"}
  ]
}
```

The SDK generates a typed library where `state.update(accel_x=9.81)` computes and accumulates the delta automatically. Change detection, checkpoint/rollback, and multi-node convergence are built in. The developer never writes XOR operations.

This matters commercially because it determines integration cost. If every customer deployment requires custom RTL work, the model doesn't scale. If the SDK handles the translation, customer engineering time is spent on their application logic, not on learning delta-state algebra. How well this holds up for production workloads is something we expect to learn from early customer integrations.

Because applications integrate through generated libraries rather than raw hardware primitives, the SDK becomes the primary developer interface to the architecture, creating a durable integration layer rather than a one-off hardware dependency.

Beachhead: Edge Embedded Systems

ATOMiK's first target market is **edge and embedded systems performing real-time sensor processing** — specifically, FPGA/SoC-based products where memory bandwidth, power efficiency, and deterministic timing are critical constraints.

Why this market first:

- These buyers already integrate third-party IP cores into FPGA/SoC designs. The evaluation and procurement motion is established.
- ATOMiK's resource footprint (287 LUT, 1.8 mW) fits within their design constraints. A 0.54% utilization impact is negligible.
- Sensor fusion is write-dominant by nature — sensor inputs arrive continuously, full state is read only at decision points. This is exactly the workload profile where ATOMiK's advantage is largest.
- These customers are more numerous, more identifiable, and more reachable than concentrated financial or defense firms.

Secondary markets (longer sales cycles, higher ACV): - **Industrial/defense real-time systems** — deterministic latency and no timing side channels are relevant to safety-critical and security-sensitive applications. 12–24 month cycles, but higher contract values. - **HFT/low-latency infrastructure** — highest-value target, but hardest to penetrate as a solo founder without warm introductions. Remains a priority but not a first-deal assumption.

I should be direct: no customers have been contacted yet. The seed capital funds both the product work and the outreach to begin these conversations. Whether these specific buyer categories convert is one of the central hypotheses the seed period tests.

Team: Addressing Solo Founder Risk

This is a legitimate concern. The solo founder model was sufficient for technical incubation — it is not the right structure for commercial execution.

What exists today: One founder who has shipped working hardware on two FPGA families, 92 machine-checked proofs, an SDK with 353 tests across 5 languages, 3 academic manuscripts, and a full production deployment — using AI-augmented development workflows that produced output equivalent to a small team over the incubation period.

Advisory board (60-day formation target, outreach beginning this week): 1. **Semiconductor/FPGA veteran** — architecture validation, foundry introductions, hiring. Target: VP/Director from AMD/Xilinx, Lattice, or Intel PSG. 2. **HFT infrastructure expert** — customer introductions in highest-value market. Target: former infrastructure lead from a quantitative trading firm. 3. **IP licensing executive** — deal structuring, pricing, sales cycle expertise. Target: BD from ARM, CEVA, or Imagination.

Compensation: 0.25–0.5% equity each, 2-year quarterly vesting. No named advisors yet — this remains a live gap.

First two hires (15% of raise, \$450K–\$600K): - **FPGA/ASIC Engineer** (Month 3–4): RTL development, Zynq deployment, ASIC feasibility. \$150K–\$200K + 1.0–2.0% equity. - **Application Engineer** (Month 5–6): SDK hardening, vertical modules, customer pilots. \$130K–\$170K + 0.5–1.5% equity.

Both sourced primarily through advisor networks once the board is seated. Without that network, this is a recruiting hypothesis, not a pipeline.

Key-person risk mitigation: The repository is structured for reproducibility — make builds hardware, lake build checks proofs, pytest runs SDK tests, flash scripts deploy in <60 seconds. Three manuscripts document the architecture. This reduces onboarding friction but does not materially remove key-person risk. The first hires and advisors are intended to begin distributing the translation burden across theory, hardware, and commercial packaging.

Revenue Model

Rebuilt bottom-up from unit economics. The original Y5 \$80M was top-down — you were right to push on it.

Commercial motion: Paid technical evaluation (\$50K–\$200K NRE) → per-design license (\$250K–\$2M) upon successful integration. Standard for semiconductor IP.

Three scenarios (Revenue = Cumulative Customers × Blended ACV):

	Y1	Y2	Y3	Y4	Y5
Conservative (single-vertical, no royalty)	\$0	\$800K	\$3M	\$7.2M	\$15M
Base (edge + industrial, 1 large deal Y4)	\$0	\$1.5M	\$6M	\$16.5M	\$35.2M
Bull (multi-vertical, ASIC licensing Y4)	\$0	\$2.5M	\$14M	\$40.5M	\$82.5M

These are planning models, not forecasts. Early revenue will be lumpy — a small number of design wins contributing the majority of initial ARR. ACV bands (\$300K–\$3M) are based on ARM/CEVA/Imagination comparables, not customer-validated pricing. The first 3–5 conversations will determine whether the right motion is low-six-figure evaluations or mid-six-figure licenses.

ASIC Strategy

The seed round does **not** fund a tapeout. It funds a feasibility study (\$150K–\$300K) — a simulation exercise producing die size, power projections, foundry selection, and a go/no-go recommendation.

Stage	Cost	Timing	Funded By
Feasibility study	\$150K–\$300K	Seed (month 9–18)	This raise
Shuttle run	\$200K–\$500K	Series A	Series A
Full tapeout (28nm)	\$2M–\$5M	Series A/B	Growth capital

Decision gates for proceeding: customer demand with committed volume (>100K units/year), completed feasibility signoff, identified design-services partner, and at least 1 co-development customer willing to share NRE. If these gates are not met, the ASIC path does not proceed. The near-term business does not require proprietary silicon — FPGA IP licensing is sufficient to validate commercial demand.

What I Still Need to Prove

Open Question	How We'll Answer It	Timeline
Will a customer pay for this IP?	Outreach to edge/embedded buyers, paid pilot	Month 6–12
Does the SDK hold up in production?	First customer integration, real workloads	Month 9–15
Can a 3-person team execute?	First 2 hires, delivery against milestones	Month 3–6
Do benchmarks transfer to real deployments?	Customer-specific workload validation	Month 9–12
Are the right advisors available?	Outreach beginning this week	Month 1–2
Does ATOMiK work on physical Zynq?	Board arriving, full PS+PL deployment	Month 1–2
Is ASIC economically viable?	Feasibility study	Month 9–18

The seed capital exists to answer these questions. The technical foundation is built. The commercial foundation is not.

Near-Term Milestones (Next 90 Days)

If funded, the first quarter of execution focuses on three goals: validating the architecture on a mainstream FPGA platform, beginning market discovery with embedded systems teams, and expanding the engineering team.

Technical - Complete physical validation on Xilinx Zynq-7020 platform - Release first public benchmark package and evaluation documentation - Extend multi-bank FPGA demonstration to N=32+

Commercial - Conduct 10–15 exploratory conversations with embedded/edge companies building sensor-processing pipelines - Identify 2–3 candidate partners for paid technical evaluation - Publish developer documentation and SDK examples

Team - Recruit and close first FPGA/ASIC engineer - Finalize advisory board (semiconductor, IP licensing, embedded systems)

These milestones are designed to test the key hypotheses behind the company: that delta-state execution provides measurable advantages in real deployments, and that embedded system teams are willing to evaluate the architecture within their existing FPGA workflows.

Appendix A: Benchmark Detail

Memory Traffic Reduction (360 measurements, Welch’s t-test p < 0.05)

Workload	Conventional	ATOMiK	Reduction	Speed
Matrix 32×32	251 MB	32 KB	7,680×	+22%
Matrix 64×64	4.0 GB	131 KB	30,720×	+22%
State Machine (100)	4.0 MB	4 KB	998×	~even
State Machine (500)	4.0 MB	4 KB	998×	~even
Streaming (5-stage)	600 KB	160 B	3,750×	+45%
Streaming (20-stage)	9.6 MB	640 B	15,000×	+58%
Scaling (16 elements)	61 KB	512 B	120×	+21%
Scaling (64 elements)	983 KB	2 KB	480×	+19%
Scaling (256 elements)	15.7 MB	8 KB	1,920×	+18%

Read/Write Crossover

Read Ratio ATOMiK vs Conventional

- 10% reads 55% faster
- 30% reads 19% faster
- ~50% reads Crossover
- 70% reads 9% slower
- 90% reads 32% slower

Hardware Measurements (Tang Nano 9K, cycle-accurate)

- Change detection: 76–80% faster than software memcmp
- Tracked memcpy overhead: +5–12% vs plain memcpy
- Operation determinism: ≤2-cycle jitter
- Parallel bank scaling: 0% deviation from linear, N=1 through N=16 (80/80 tests)
- Peak throughput: 1,056 Mops/s (16 banks at 66 MHz)

Appendix B: XOR and Data Types

XOR operates on bit patterns, not data types. It handles integers, IEEE 754 floats, packed structs, and raw binary identically — same hardware path, same cycle count (192 round-trip), same energy (16.4 nJ). This is not parity: the accumulator preserves full bit-width (64 bits), enabling exact state reconstruction. Parity reduces N bits to 1 bit and cannot reconstruct.

Limitations: fixed-width operands only (variable-length data segmented by SDK), not arithmetic (state tracking, not computation on deltas), and read-heavy workloads pay a reconstruction penalty.

Appendix C: Available Materials

- Raw benchmark CSV data (1,164 rows across 4 datasets)
- Synthesis logs and resource reports (Gowin and Vivado)
- Hardware test results (80/80 parallel bank sweep, 9/9 ATOMiK instruction tests)
- Lean4 proof source files (92 theorems, machine-verifiable)
- SDK test reports (353 tests, 5 languages)
- Academic manuscripts (3, LaTeX source and PDF)
- Zynq synthesis artifacts and build scripts

Looking forward to Thursday. Happy to walk through any of this in detail.

Best, Matt